

Работа с gpio в режиме выхода

Документация:

- <https://origin.kernel.org/doc/html/latest/driver-api/gpio/driver.html>
документация по GPIO"

"Официальная

Практическая часть реализуется при работе с платой **LicheeRV Nano**

1. Определяемся кто есть кто

1.1. Смотрим доступные порты

```
# ls /sys/class/gpio/
export      gpiochip384  gpiochip448  unexport
gpiochip352 gpiochip416  gpiochip480
```

Мы видим **gpiochipXXX** и она привязана к адресу порта, сам пин определяется смещением относительно базового, то есть нулевого пина. Количество пинов до 32 (смещение от 0 до 31). То есть пин $\text{GPIOX}_{15} = \text{gpiochipXXX} + 15$

2. Определяемся с портом

Выполняем следующий код, мы получили адрес,

```
# cat /sys/class/gpio/gpiochip480/label
3020000.gpio
```

далее зайдя в руководство на процессор и сопоставив информацию, получаем четкое понимание (пример из руководства на SG2002)

21.5.3 GPIO Register Overview

The chip includes 4 groups of GPIO under Active Domain and 1 group of GPIO under No-die Domain. The base address is shown in table *Base addresses of GPIO modules*.

Table 21.140: Base addresses of GPIO modules

GPIO Module	Base Address
GPIO0	0x03020000
GPIO1	0x03021000
GPIO2	0x03022000
GPIO3	0x03023000
RTCSYS_GPIO	0x05021000

Table *GPIO Registers Overview* is the offset address and definition of the registers of the GPIO module (GPIO0) in group 1. GPIOs in other groups have the same register definitions.

21.5 GPIO

The system includes 4 groups of GPIO (General Purpose Input/Output) under Active Domain, which are GPIO0 ~ GPIO3, and 1 group of GPIO under No-die Domain, RTCSYS_GPIO. Each group of GPIO provides 32 programmable input and output pins.

Note: In this manual, GPIOA is often used instead of GPIO0, GPIOB instead of GPIO1, GPIOC instead of GPIO2, and GPIOD instead of GPIO3.

The direction of each pin can be arbitrarily set as input or output, used to generate output signals for specific applications or collect input signals for specific applications. When set as an input pin, the GPIO can be used as an interrupt source; when set as an output pin, each GPIO can independently output 0 or 1.

GPIO can generate maskable interrupts based on the level or transition value of the input signal. The GPIOx_INTR_FLAG (x = 0 ~ 3) signal gives the interrupt controller an indication that an interrupt has occurred.

3. Использование как выхода

3.1. Теперь пробуем переключать используем устаревший способ

```
1. "Экспортируем" пин 495 (сообщаем ядру, что будем его использовать)
Если пин уже используется, эта команда выдаст ошибку "Device or resource busy"
# echo 495 | tee /sys/class/gpio/export
495
2. Настраиваем пин как ВЫХОД (out)
# echo out | tee /sys/class/gpio/gpio495/direction
out
3. Записываем значение 0 (LOW / выключить)
# echo 0 | tee /sys/class/gpio/gpio495/value
0
4. Записываем значение 1 (HIGH / включить)
# echo 1 | tee /sys/class/gpio/gpio495/value
1
5.
echo 495 | sudo tee /sys/class/gpio/unexport
```



1. Для текущего варианта записи может понадобится sudo
2. Также возможно задавать в таком синтаксисе, но необходим root

```
echo 1 > /sys/class/gpio/gpio495/value
```

и так у меня один пин под номером 15 работает, а 17 не управляется но он судя по схеме еще может быть задействован как uart0, что очень вероятно и как pwm5, при этом вывод в

консоль у обоих пинов аналогичен.

4. Использование пина как входа

Данный код я не проверял Разбор отличий:

- **direction:** Вместо out мы пишем in. Это физически переключает внутреннюю схему контроллера на прием сигнала.
- **Чтение (cat):** Вместо того чтобы записывать (echo) значение в файл value, мы его читаем с помощью команды cat. Если на пине высокий уровень (HIGH), ты увидишь 1, если низкий (LOW) — 0.

```
# 1. "Экспортируем" пин 495 (сообщаем ядру, что будем его использовать)
echo 495 | sudo tee /sys/class/gpio/export

# 2. Настраиваем пин как ВХОД (in)
echo in | sudo tee /sys/class/gpio/gpio495/direction

# 3. Читаем текущее значение пина (вернет 0 или 1)
cat /sys/class/gpio/gpio495/value

# ... можно читать много раз, например, в цикле ...

# 4. Освобождаем пин (обязательно!)
echo 495 | sudo tee /sys/class/gpio/unexport
```

Но нельзя делать бесконечный цикл с cat для опроса кнопки. Это "съест" 100% процессорного времени. В Linux GPIO есть механизм прерываний (edge detection). Можно сказать ядру: "Разбуди мою программу только тогда, когда сигнал изменится". Для этого используется файл edge:

```
# Доступные значения для edge:
# none - прерывания отключены (по умолчанию)
# rising - сработает при изменении с 0 на 1 (нажатие кнопки)
# falling - сработает при изменении с 1 на 0 (отпускание кнопки)
# both - сработает при любом изменении

# Настраиваем пин на реакцию на восходящий фронт (0 -> 1)
echo rising | sudo tee /sys/class/gpio/gpio495/edge
```

После настройки edge, в языках программирования (C, Python) или через утилиты (например, inotifywait в bash) ты можно эффективно "ждать" изменения сигнала, не нагружая процессор.

```
# cat /sys/kernel/debug/gpio 2> /dev/null
gpiochip4: GPIOs 352-383, parent: platform/5021000.gpio, 5021000.gpio:
```

```

gpio-353 (          |snsr-rst-gpio      ) in  lo
gpio-355 (          |GTP INT IRQ        ) in  lo IRQ
gpio-356 (          |GTP RST PORT       ) out  hi

gpiochip3: GPIOs 384-415, parent: platform/3023000.gpio, 3023000.gpio:

gpiochip2: GPIOs 416-447, parent: platform/3022000.gpio, 3022000.gpio:

gpiochip1: GPIOs 448-479, parent: platform/3021000.gpio, 3021000.gpio:
gpio-449 (          |scl                ) out  hi
gpio-450 (          |sda                ) in   lo
gpio-454 (          |cviusb-otg         ) in   hi IRQ

gpiochip0: GPIOs 480-511, parent: platform/3020000.gpio, 3020000.gpio:
gpio-493 (          |cvi-cd             ) in   lo IRQ ACTIVE LOW
gpio-494 (          |led-user           ) out  lo
gpio-510 (          |User Key           ) in   hi IRQ ACTIVE LOW

```

Таким образом мы видим пин **gpio-494** который задействован драйвером led. Пытаемся управлять им напрямую, по старому протоколу в последних версиях ядро это осуществляется через библиотеку

```

# echo 494 > /sys/class/gpio/export
sh: write error: Resource busy
# echo out > /sys/class/gpio/gpio494/direction
-sh: can't create /sys/class/gpio/gpio494/direction: nonexistent directory

```